**GEBZE TECHNICAL UNIVERSITY**

Computer Engineering Department

CSE 396 Module Documentation

# Çılgın Kurye

*Module M3: Physics and Collision*

**Instructor:** Erkan Zergeroğlu **Company:** Board&Byte **Group:** Group-8 **Date:** 6 November 2026



| Name | Student No. | Name | Student No. |
|---|---|---|---|
| Abdullatif Yılmaz | 230104004031 | Hasan Alperen Elmas | 1801042604 |
| Batuhan Yanar | 220104004059 | Selim Barut | 220104004046 |

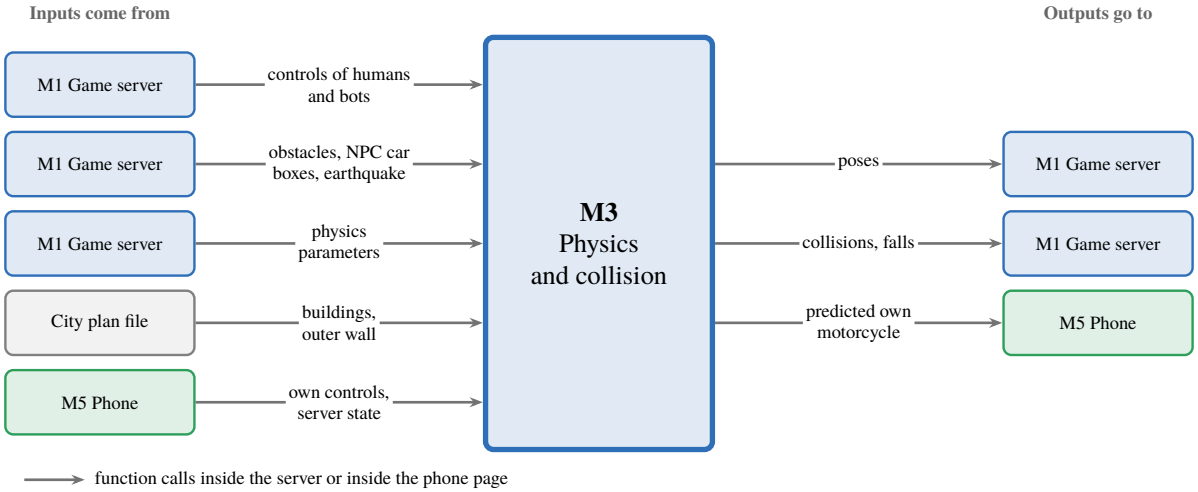


## 1. Purpose

M3 moves the motorcycles. It turns throttle, brake and steering into speed, direction and position. It finds collisions, decides when a motorcycle falls, and adds the effect of obstacles and earthquakes to the motion. The same code runs in two places: on the server, where its result is the real one, and on the phone, where it guesses the player's own motorcycle before the server answers.

| Runs on | Technology | Requirements |
|---|---|---|
| Main computer inside the game server, and the phone browser for prediction only | TypeScript library, our own simple 2D arcade physics without a ready physics engine, circle and box collisions on a uniform grid, seeded random numbers | R3 R4 R6 R7 R11 |

M3 has no network code. On the server it talks only to M1, and on the phone only to M5. The figure shows both uses.




## 2. Requirements covered

M3 takes part in five requirements. The last column says what will work when M3 is finished.

| ID | Requirement | What M3 does for it | When M3 is done |
|---|---|---|---|
| R3 | Driving and controls | Throttle, brake, left and right. No reverse. A U turn by steering in a small circle at low speed | The motorcycle drives, brakes and turns like an arcade game |
| R4 | Collisions and falls | Collisions with buildings, barriers, NPC cars and motorcycles. A fast hit makes the motorcycle fall | Nothing drives through a wall, and fast crashes end in a fall |
| R6 | Obstacles | Road work and accidents are solid. An accident has a slow zone. A puddle makes the motorcycle slide | Each obstacle type feels different when you drive |
| R7 | Earthquake | Side push and less grip inside the earthquake radius that M1 sends | Riding in an earthquake is harder |
| R11 | Engineering requirements | The same input always gives the same result. The server result wins, the phone only predicts | The own motorcycle reacts at once and still matches the server |

# 3. Inputs and outputs

M1 calls M3 two times in every tick with the inputs below. M5 uses the same code for one motorcycle.

| Direction | Data | From / to | Content |
|---|---|---|---|
| In | City plan | Shared file | Roads, lanes, blocks and buildings as fixed colliders |
| In | Controls | M1 | Every tick: throttle and brake from 0 to 1, steer from −1 to 1, for humans and bots, and a speed factor for each player |
| In | Obstacles, NPC cars | M1 | Obstacle type, road, lane, position, and the boxes of the NPC cars that M1 gets from M4 |
| In | Earthquake | M1 | Centre, strength, radius, and the players that fall |
| In | Parameters | M6, through M1 | Top speed, grip, fall speed, puddle and earthquake effects |
| Out | Poses | M1 | Position, direction, speed, lean, sliding, fallen |
| Out | Events | M1 | Collisions (with what and how fast) and falls. M1 gives the car hits to M4 |
| Out | Predicted pose | M5 | The player's own motorcycle before the server answers |

# 4. Main components

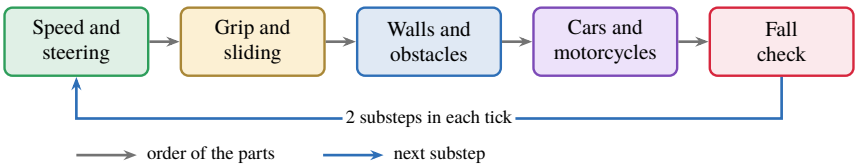
M3 has five parts. The first four run on the server, the last one runs on the phone.

| Component | Takes | Gives |
|---|---|---|
| Motorcycle model | Controls, parameters | New speed, direction, lean and sliding |
| Colliders | City plan, obstacle list, NPC cars | Boxes, slow zones and slippery areas in a uniform grid |
| Collision solver and fall judge | Moved objects, colliders | Pushed apart objects, collision and fall events |
| Earthquake effect | Earthquake centre, strength, radius | Side push and lower grip inside the radius |
| Predictor | Own controls, server state | Predicted own motorcycle with a smooth correction |

# 5. How it works

**One physics step:**

The server runs 30 ticks per second, and M3 runs 2 substeps in each tick. Small steps keep fast motorcycles from passing through thin walls. Each substep does five things in a fixed order, as the figure shows. A fixed order and seeded random numbers mean that the same inputs always give the same result on the server and on the phone.

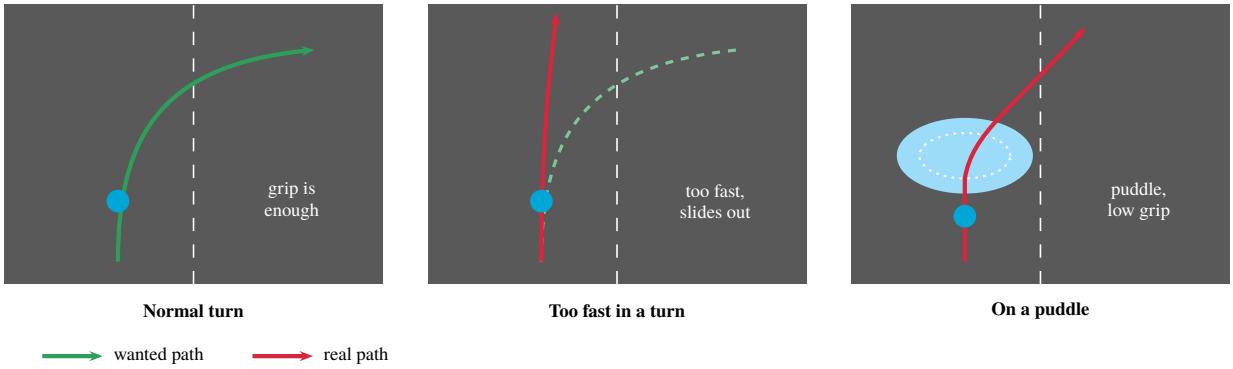



**Driving:**

The throttle adds speed up to the top speed, and the brake and air drag take speed away. There is no reverse. The steering changes the direction. At low speed the motorcycle can turn in a small circle, so a U turn is easy. At high speed the turn is wider. The model also gives a lean angle, which M5 uses to tilt the 3D motorcycle in a turn.

**Grip and sliding:**

In a turn the motorcycle needs a side force. M3 compares the size of this force with the grip limit. If the force is below the limit, the motorcycle follows the turn. If it is above the limit, the motorcycle slides out of the turn. We compare only the size, so a left turn and a right turn act the same. On a water puddle the grip limit is very low, so the motorcycle slides but does not slow down. Inside an earthquake radius the grip is lower too, and there is a small push to the side.

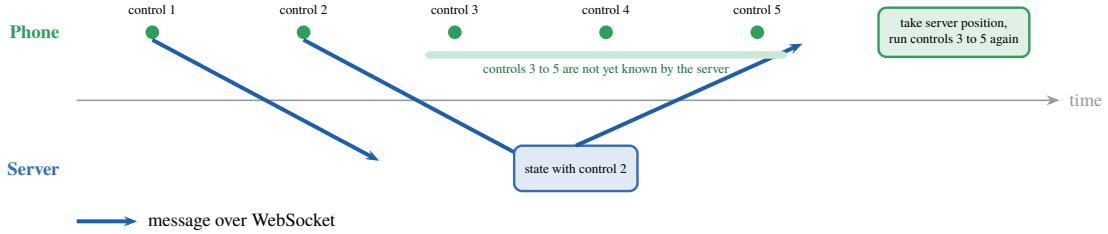



**Collisions and falls:**

Buildings, the outer wall and the road work barriers come from the city plan and the obstacle list as boxes. NPC cars are boxes, and motorcycles are circles. To find collisions fast, M3 puts every object into a uniform grid, so it checks only objects in the same or next cell. A contact pushes the objects apart. If the hit speed is above the fall speed that the admin set, the motorcycle falls. When two motorcycles hit each other and their relative speed is above the limit, both of them fall. A fallen motorcycle is not solid, so it does not block the road for others. M1 gives the penalty time and later asks M3 to stand the motorcycle up again. An accident is solid in one lane and has a slow zone around it.

**Prediction on the phone:**

On the phone, the player must see the own motorcycle move at once, not after the delay of the network. So M5 runs a copy of M3 for this one motorcycle. Each control gets a number and is kept in a list. When a server state arrives, the phone takes the real position from the server, throws away the controls that the server has already used, and runs the remaining controls again. If the new result is close to what the player sees, the difference is blended away in a short time. If it is far, for example after a crash, the motorcycle jumps to the right place.




# 6. Testing

M3 needs no other module. A test makes a world from the city plan and gives scripted controls, fixed obstacles and fixed NPC car boxes as test data. We check a straight run and braking, a U turn, slides to the left and to the right, wall and car crashes, the earthquake radius, and that the same inputs give exactly the same result two times. The predictor is tested with recorded server states and added delay and loss. A small test page shows a simple 2D view of the city, so one motorcycle can also be driven with the keyboard and the parameters can be changed with keys.